

PANES : Policy-guided Asymmetric Nash Equilibrium Search

Youhan Ialwani

*Department of Artificial Intelligence
and Data Science
Thadomal Shahani Engineering
College
Mumbai, India
youhanlalwani@gmail.com*

Mahesh Patel

*Department of Artificial Intelligence
and Data Science
Thadomal Shahani Engineering
College
Mumbai, India
patelbhiyaram757@gmail.com*

Parthavi Gaikwad

*Department of Artificial Intelligence
and Data Science
Thadomal Shahani Engineering
College
Mumbai, India
pgaikwad0601@gmail.com*

Astha Mishra

*Department of Artificial Intelligence
and Data Science
Thadomal Shahani Engineering
College
Mumbai, India
asthamishra1905@gmail.com*

Dr. Sanobar Shaikh

*Department of Artificial Intelligence
and Data Science
Thadomal Shahani Engineering
College
Mumbai, India
sanobar.shaikh@thadomal.org*

Abstract—Trick-taking card games with mandatory bidding confront reinforcement learning agents with a distinctive two-phase problem: each player must commit to a numeric bid before any cards are played, and whether that bid turns out to be correct hinges on adversarial interactions unfolding over many subsequent tricks. Judgement (also called Oh Hell) makes this harder still through its Hook rule, which structurally prevents the total bids from summing to the available tricks. This ensures at least one player is guaranteed to fail every round. Here, we introduce a complete framework of reinforcement learning for the Judgement game, consisting of an environment and a specific agent architecture trained together in bidding and trick-play under imperfect information. The environment was built inside the RLCard platform, supporting four-player games with legal-action masking and lightweight state rollback for tree search. On top of this, we developed a PANES agent that pairs Neural Fictitious Self-Play with Information Set Monte Carlo Tree Search. During training, the agent learns through self-play guided by dense trick-level reward shaping; at decision time, it runs multi-step lookahead over determinized game worlds using the learned opponent policies. In a 2,000-game paired-deal arena evaluation, the PANES agent reached 22.7% bid accuracy versus 13.5% for the neural-only baseline—a 68% relative gain, supported by robust statistical significance ($p < 0.0001$). A closer look at the training dynamics revealed an interesting asymmetry: agents pick up trick-winning skills quickly but struggle much more with deliberate trick avoidance, which we identify as the main learning bottleneck. We also ran an ablation with Prioritized Experience Replay; it cut training loss by roughly a third yet produced no measurable improvement in game-level performance, pointing to the credit assignment horizon as the binding constraint. Taken together, these findings show that merging equilibrium-based self-play with online tree search is a viable path for imperfect-information trick-taking games, while the long-horizon credit assignment problem remains the central open challenge.

Index Terms—Reinforcement Learning, Monte Carlo Tree

Search, Neural Fictitious Self-Play, Imperfect-Information Games, Trick-Taking Card Games, Nash Equilibrium, Multi-Agent Systems, Dense Reward Shaping

I. INTRODUCTION

Reinforcement learning has come a long way in game playing, but partially observable environments (which have game players with private information) are inherently harder to solve. Card games are a prime example: each hand is private, there's long horizons between actions, and the optimal move depends on the other player's hidden state. Although deep RL has been successful for other trick-based card games, e.g., DouZero [1], PerfectDou [2], these programs are either game-specific or require the oracle teaching of the current trick. Other trick-taking card games that include forced bidding is largely uncharted territory.

Judgement (also called Oh Hell) is an example. After players have bid how many tricks they think they will win, players will proceed with an adversarial game of 13 tricks, playing cards back and forth between them. The unique Hook rule prevents the dealer from bidding just that many tricks to the total sum of tricks players believe they will win, creating a game where someone will always lose. Equally important, overbidding and underbidding carry the same penalty, forcing agents to learn not just how to win tricks but also when to deliberately lose them.

Judgement is difficult for standard RL due to 52 actions separating the first action and terminal reward [3]. NFSP [4] learns equilibrium strategies but only looks 1 step ahead, while MCTS [5] looks ahead but doesn't rely on learned knowledge.

We address both gaps by extending NFSP self-play with extensive reward shaping and void-aware determinization, building on the work of Zhang et al. [6] and propose **PANES** (Policy-guided Asymmetric Nash Equilibrium Search) for trick-taking card games. Our contributions are :

- 1) A complete Judgement environment within RLCard [7], with a 454-dimensional state space, 66-action discrete space, and lightweight checkpoint support for tree search.
- 2) A dense reward framework for the 13-trick credit assignment problem, providing trick-level bid-alignment signals on top of the sparse terminal score.
- 3) The PANES agent: asymmetric IS-MCTS with MRV void-respecting determinization, and NFSP-guided opponent simulation.
- 4) An empirical evaluation showing 22.7% bid accuracy for PANES vs. 13.5% for the pure neural baseline—a 68% relative gain—over 2,000 paired-deal games ($p < 0.0001$).

II. RELATED WORK

There are two types of algorithms we can use to address sequential imperfect information decision-making: equilibrium-finding algorithms and online search. Fictitious play [8], [9] and its deep counterpart, Neural Fictitious Self-Play (NFSP) [4], learn to approximate Nash equilibria in large games via a combination of a best-response DQN [10] and a supervised average policy network. NFSP can avoid cycle exploitation, but the one-step TD backups used in NFSP makes the learned equilibria unsuitable for long-horizon tasks.

Conversely, Information Set Monte Carlo Tree Search (ISMCTS) [5], [11] learns strong multi-step lookahead strategies for games by building search trees over determinized states of the game. It is limited, however, by building a new search tree for each decision, it suffers from strategy fusion [12] in high-uncertainty state spaces, and it does not learn to transfer strategic knowledge across games.

To address this shortcoming, Zhang et al. [6] later proposed MC-NFSP, with the single-step Q-value estimate replaced by MCTS rolls under the evaluation of average policy. While this results in a strong two-player poker system, transitioning search-learning integration to multi-player trick-taking games such as Judgement introduces distinct challenges: 1) opponent behavior must be well simulated under forced following rules, requiring void-aware determinization, 2) search trees expand exponentially with the number of players. In-context opponent modeling [13] suggests it could be beneficial but it comes at a significant inference overhead. Our proposed PANES tackles these issues through an asymmetric tree search where the opponents are modeled by learned policies.

Propagating reward signals over long trick sequences is another major challenge. Potential-based reward shaping [14] guarantees policy invariance but it requires hand-crafted potential functions. Alternative methods like Prioritized Experience Replay (PER) [15] prioritize informative states but do not shorten the horizon. Algorithms such as DouZero [1] and

PerfectDou [2] use Monte Carlo returns or oracle-guided value networks to handle sparse signals, but they either suffer from high variance or require parallel training of privileged models. In contrast, our dense trick-level schedule guides bidding and play alignment dynamically without auxiliary networks. Finally, while platforms like RLCard [7] and OpenSpiel [16] standardize card game evaluation, neither supports the multi-stage bidding and play dynamics of Judgement, a gap our environment fills.

III. THE JUDGEMENT GAME ENVIRONMENT

Building an RL environment for Judgement means turning the informal rules of a card game into a precisely specified decision process – without losing the structural properties that make the game strategically interesting. We kept three design principles in mind throughout: (1) all publicly available information should be fully observable, with private hands properly masked; (2) rule enforcement should live entirely inside the environment, with no game-rule knowledge leaking into agent code; and (3) game-state serialization must be efficient enough to support the many rollbacks required by MCTS. The resulting implementation is integrated into the RLCard framework [7], making it the first trick-taking game supported by that platform.

A. Formal Problem Formulation

We define Judgement as a four-player imperfect information extensive-form game, following the Markov game framework [17]. The decision problem faced by each player is modeled as a Partially Observable Markov Decision Process (POMDP) defined by the tuple $\langle \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma, \mathcal{P}_0 \rangle$. Here, $\mathcal{S}$ represents the set of all possible world states (deck distribution, private hands, bids, trick history); $\mathcal{O} \subset [0, 1]^{454}$ is the local observation space of each player, masking private opponent hands; $\mathcal{A}$ is the 66-dimensional action space with a legal-action mask; $\mathcal{T}$ represents transition dynamics; $\mathcal{R}$ is the reward function; $\gamma = 0.995$ is the discount factor; and $\mathcal{P}_0$ is the initial uniform dealing distribution. The training objective is to find a joint policy profile $\pi = (\pi_1, \dots, \pi_4)$ approaching a Nash equilibrium via Neural Fictitious Self-Play [4].

B. Game Rules and Action Space

Judgement is a 52-card game for four players. In our training setup we fix hand size to $n = 13$ tricks, with the trump suit rotating deterministically ($\spadesuit \rightarrow \diamondsuit \rightarrow \clubsuit \rightarrow \heartsuit$) each round to prevent overfitting. Each round has two phases: (1) *Bidding Phase* (Actions 0–13): players sequentially declare a predicted trick count $b_i \in \{0, \dots, n\}$, with the dealer bidding last under the Hook constraint ($\sum b_i \neq n$), guaranteeing that at least one player fails to meet their contract. (2) *Playing Phase* (Actions 14–65): the player to the dealer’s left leads the first trick; subsequent players must follow suit if able, otherwise discarding or trumping, and the trick is won by the highest card of the led suit or the highest trump played. The action space consists of 66 distinct actions spanning these two phases.

TABLE I
OBSERVATION SPACE DECOMPOSITION (454 FEATURES)

Component	Dim.	Encoding
Player’s Hand	52	Multi-hot bitmask
Trump Suit	4	One-hot indicator
Agent’s Own Bid	14	One-hot (0–13)
Other Players’ Bids	$3 \times 14 = 42$	One-hot per player
Tricks Won (all players)	$4 \times 14 = 56$	One-hot per player
Current Trick Cards	$4 \times 52 = 208$	Per-seat card slot
Played Cards History	52	Cumulative multi-hot
Round Card Count	14	One-hot (1–13)
Phase Indicator	1	Binary (bid/play)
Current Player	4	One-hot indicator
Round Progress	1	Fraction round_idx/total
Hand Strength Heuristics	6	Continuous [0, 1]
Total	454	

The environment provides a legal-action mask at every step to enforce the rules.

Instead of using the standard trick-scoring convention $\pm(10 + b_i)$, which incentivizes bid inflation, we use a binary scoring system:

$$r_i = \begin{cases} +1.0 & \text{if } w_i = b_i, \\ -1.0 & \text{if } w_i \neq b_i, \end{cases} \quad (1)$$

where w_i is the number of tricks won.

The game state is encoded as a 454-dimensional vector of values in $[0, 1]$ to bound gradient norms, detailed in Table I. Cards are mapped to $\{0, \dots, 51\}$ via suit $\times 13 + \text{rank}$. Instead of a flat history, we use four separate 52-bit fields to track which player played each card in the current trick, preserving temporal and seating correlations. To accelerate early learning, we append 6 hand-strength features representing the ratio, high-card count, and average rank of trump and non-trump suits in the agent’s hand.

IV. METHODOLOGY

Our approach unfolds in two stages. We first train a population of NFSP agents purely through self-play, which leaves each of the four players with both a best-response Q-network and an average-policy network. Once training has converged, we wrap every trained agent in a PANES shell that runs Information Set MCTS at inference time, using the learned policies to simulate opponents and to score leaf states during search.

A. NFSP Training Architecture

Each player runs an independent NFSP agent [4], and all four share the same multilayer perceptron:

$$\text{Linear}(454) \rightarrow 1024 \rightarrow 512 \rightarrow 256 \rightarrow 66 \quad (2)$$

with ReLU activations, trained with Adam [18] at a learning rate of 5×10^{-4} : the Q-network against MSE loss over bootstrapped targets, the average-policy network against NLL loss over supervised action distributions. Agents act through an anticipatory mixing strategy – greedy best response with

probability $\eta = 0.15$, the average policy otherwise – which keeps the population from cycling into exploitable patterns during multiplayer self-play. The Q-network itself follows a Double DQN scheme with target refreshes every 500 steps and ϵ -greedy exploration decayed from 1.0 to 0.1 over 2.1×10^6 steps, drawing batches of 512 from a 350,000-capacity replay buffer, while the average-policy network trains on a reservoir buffer of the same size to preserve a representative record of past play.

B. Dense Reward Shaping

Credit assignment over a 13-trick, 52-action horizon is unstable under the sparse terminal payoff (± 1.0) alone, so we layer two dense signals on top of it. During bidding, a hand-strength heuristic estimates the expected trick count $\hat{T}$ from the hand’s ranks and trump distribution, and the agent is rewarded for bidding close to that estimate:

$$r_{\text{bid}} = \max\left(-1.0, 0.5 - 0.3|b - \hat{T}|\right). \quad (3)$$

During play, each resolved trick is scored by whether it moves the agent’s running trick count closer to or further from its bid – rewarding trick wins while a bid is still outstanding and ducks once it is met, and penalizing the reverse, with a heavier penalty for overshooting than for falling short, since over-winning is the harder failure mode to unlearn. These per-trick scores accumulate into a running total $D_t^{(i)}$, so the reward on any transition is simply $r_t = D_{t+1}^{(i)} - D_t^{(i)}$, with the binary terminal payoff folded in at the last step. The result is one continuous target that blends trick-by-trick feedback with the true game outcome, rather than asking the agent to learn everything from a single payoff thirteen tricks later.

C. PANES Agent

At inference time, each trained agent runs Information Set MCTS over determinized game states (Fig. 1). Since opponent hands are hidden, every simulation first reconstructs a plausible deal: known cards (the agent’s hand, exposed trump, and played tricks) are removed from the deck, suit-following failures in the trick history are used to infer void constraints, and the remaining unknown cards are dealt to opponents ordered by how constrained they are, so void rules are respected without redeals. Rather than branch the full four-player tree – which grows combinatorially – the search tree is asymmetric: only the agent’s own actions become nodes, while opponent turns are resolved by sampling directly from their trained average-policy network π_{avg} . Node selection follows the standard UCB1 rule [19], and search is capped at a shallow depth of $D = 2$, covering two rounds of the agent’s own decisions.

Leaf nodes are scored with a blend of the learned Q-value and a bid-alignment heuristic,

$$V_{\text{leaf}} = 0.7 Q_{\text{DQN}}(s) + 0.3 H_{\text{bid}}(s), \quad (4)$$

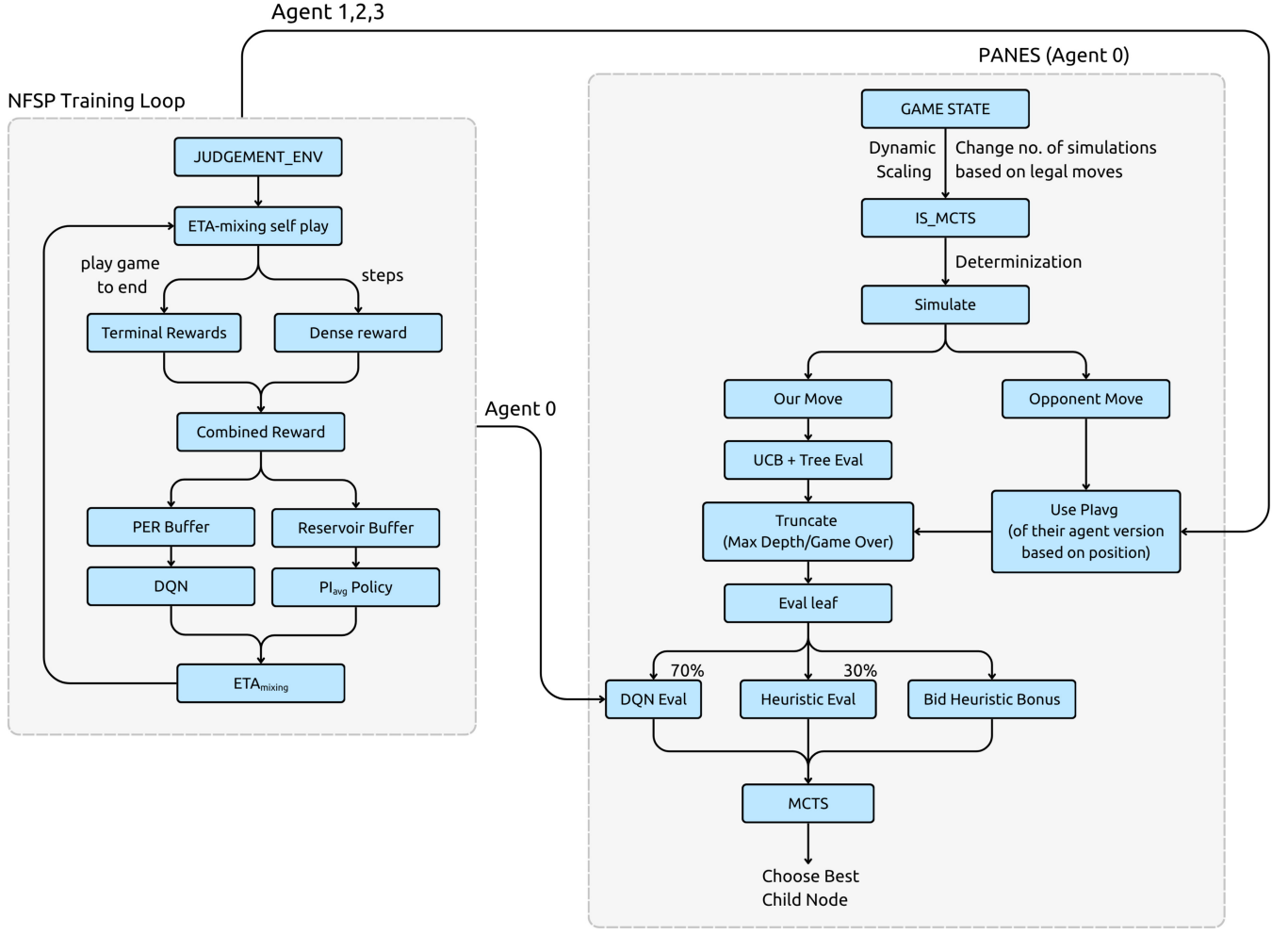


Fig. 1. The PANES architecture. Left: autonomous self-play populates the replay buffer for Q-network updates and the reservoir buffer for average-policy updates. Right: at inference, the trained networks guide Information Set MCTS as both an opponent-move simulator and a leaf-evaluation heuristic.

where, letting $g = b_i - w_i$ be the tricks still needed and k the tricks remaining,

$$H_{\text{bid}}(s) = \begin{cases} 0.3 + 0.4(1 - k/n), & g = 0 \text{ (bid met)}, \\ 0.1(1 - g/k), & g > 0, k \geq g, \\ -0.5, & g > 0, k < g \text{ (impossible)}, \\ \text{clip}(-0.3|g|, -1, 0), & g < 0 \text{ (exceeded bid)}. \end{cases} \quad (5)$$

This steers the search toward the exact contract while terminal states still fall back on the official binary score. Finally, because a narrow set of legal actions carries more determinization noise, the simulation budget scales up dynamically in those cases – up to $4\times$ the base budget of $N_{\text{sim}} = 200$ when three or fewer actions are legal – so search effort concentrates where uncertainty is highest. Algorithm 1 summarizes the procedure.

Algorithm 1 PANES Decision Procedure

Require: Observation s , legal actions $A(s)$, base budget N_{sim} , depth limit $D = 2$

Ensure: Selected action a^*

- 1: $N \leftarrow$ budget scaled by $|A(s)|$ {Section IV-C}
 - 2: **for** $i = 1$ **to** N **do**
 - 3: $s' \leftarrow \text{DETERMINIZE}(s)$ {MRV void-respecting deal}
 - 4: **Select:** descend via UCB1 while $d < D$; agent turns branch, opponent turns sample $a_{\text{opp}} \sim \pi_{\text{avg}}(\cdot | s')$
 - 5: **Expand** the resulting leaf with all $a \in A(s')$
 - 6: **Fast-forward** remaining opponent turns via π_{avg} until terminal or the agent's turn
 - 7: $V_{\text{leaf}} \leftarrow 0.7 Q_{\text{DQN}}(s') + 0.3 H_{\text{bid}}(s')$
 - 8: **BACKPROPAGATE**($\text{clip}(V_{\text{leaf}}, -1, 1)$) up the tree
 - 9: **end for**
 - 10: **return** $a^* \leftarrow \arg\max_{a \in A(s)} N(v_0 \rightarrow a)$
-

V. EXPERIMENTAL SETUP

A. Training Configuration

All four NFSP agents train from random initialization via self-play on isolated 13-card rounds for 600,000 episodes with the Adam optimizer [18]; Table II lists the key hyperparameters.

TABLE II
TRAINING HYPERPARAMETERS

Parameter	Value
Network architecture	454 $\rightarrow$ 1024 $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$ 66
Learning rate (both networks)	5×10^{-4}
Discount factor γ	0.995
ε -decay steps	2.1×10^6 (150,000 episodes $\times$ 14 steps)
Anticipatory parameter η	0.15
Batch size	512
Replay buffer capacity	350,000
Reservoir buffer capacity	350,000
Target network update interval	500 steps
Training frequency	Every 32 steps

B. Evaluation Pipeline

The agents are evaluated using a four-phase pipeline.

Phase 1 – NFSP Training logs taken 500 episodes.

Phase 2 – PANES Evaluation evaluates four Hybrid agents with exploration disabled ($\varepsilon = 0$).

Phase 3 – Pure NFSP Baseline runs the agents deterministically ($\eta = 1$).

Phase 4 – Arena (PANES vs. Pure) pits a single Hybrid agent against three Pure NFSP opponents.

In the last phase we utilize paired dealings. Each deal is played twice, once with the PANES Agent and once with a Pure NFSP agent to ensure that the change in win rate is isolated from positional effects and card luck.

C. Metrics

We track three metrics throughout: *average payoff*: the mean binary terminal reward (± 1.0) across evaluation games; *bid accuracy*: (“Won”), underbids (“Under”), or overbids (“Over”); and *training loss trajectories*: MSE for the Q-network and NLL for the average-policy network.

VI. RESULTS

A. Arena Evaluation: PANES vs. Pure NFSP

In Phase-4 we pit a single PANES agent against three Pure NFSP opponents across 2,000 paired deals (500 per seat). Each deal is played twice. Once with the Hybrid in the seat and once with a Pure NFSP agent under identical card distributions to isolate the effect of MCTS lookahead from positional and card-luck variance.

As shown in Table III and Figure 2, the PANES agent achieves 22.7% bid accuracy versus 13.5% for Pure NFSP. This is a +9.2 pp (68% relative) gain, placing it at $2.9\times$ the random baseline. The advantage is consistent across all seats (Figure 3), with the largest swing in Seat 1 (+10.8 pp).

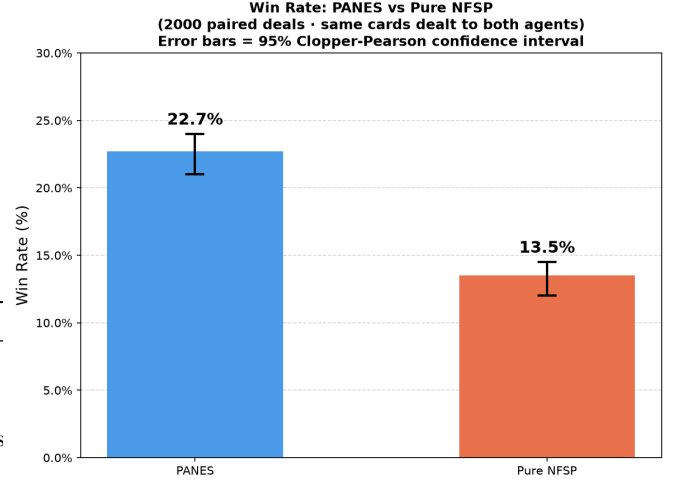


Fig. 2. Overall Arena Evaluation comparing 2,000 paired deals.

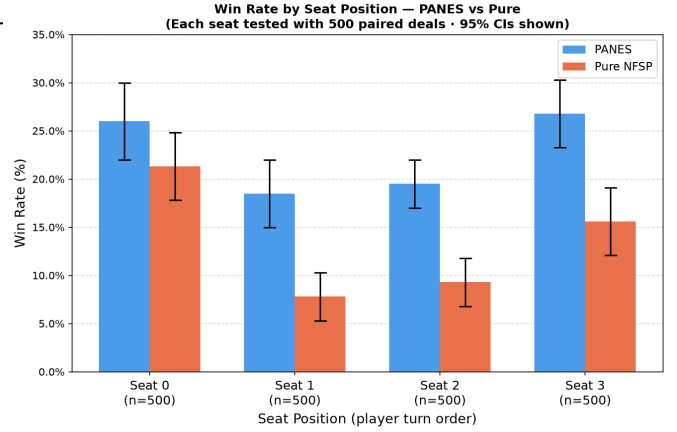


Fig. 3. Win rate grouped by starting seat position

B. Statistical Validation

McNemar analysis (Figure 4) on the 2,000 paired deals shows 305 Hybrid-exclusive wins versus 120 Pure-exclusive wins ($\chi^2 = 79.66$, $p < 0.0001$). A 10,000-resample bootstrap (Figure 5) gives a 95% CI of $[+7.3\%, +11.2\%]$ confirming the advantage is not caused by card-luck variance.

Training metrics are logged every 500 episodes, averaged across all seats, and smoothed with a 10,000-episode rolling window. The PER ablation is in Section VI-G.

C. Training Convergence

Figure 6 shows three training phases: an *exploration phase* (0–100K) near the 7.7% random baseline; a *rapid learning phase* (100K–250K) where payoff climbs from -0.84 to -0.79 and accuracy reaches $\sim 10\%$; and a *plateau phase* (250K–600K) converging to 12.1% accuracy—a 57% improvement over random, consistent with approximate Nash equilibrium.

Table IV tracks progress at key checkpoints in the training process. Game outcomes improve monotonically even though

TABLE III
ARENA RESULTS: PANES VS. PURE NFSP (2,000 PAIRED GAMES)

Seat	PANES Win %	Pure Win %	Δ Win %
Random Baseline	N/A	N/A	7.7% (Theoretical)
Seat 0	26.0%	21.2%	+4.8 pp
Seat 1	18.6%	7.8%	+10.8 pp
Seat 2	19.4%	9.4%	+10.0 pp
Seat 3	26.6%	15.6%	+11.0 pp
Average	22.7%	13.5%	+9.2 pp

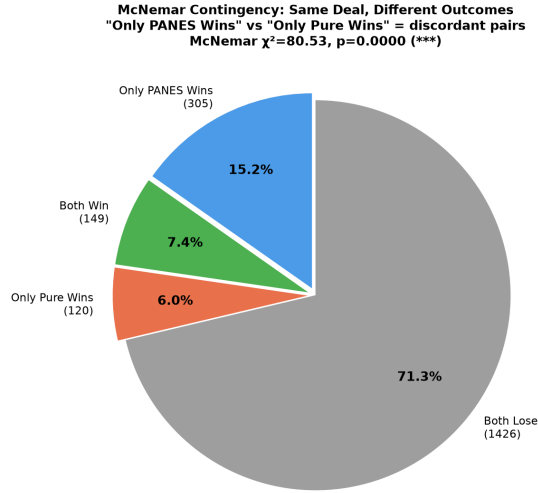


Fig. 4. McNemar contingency breakdown highlighting discordant game pairs. The χ^2 value of 79.66 ($p < 0.0001$) indicates a significant difference in capability.

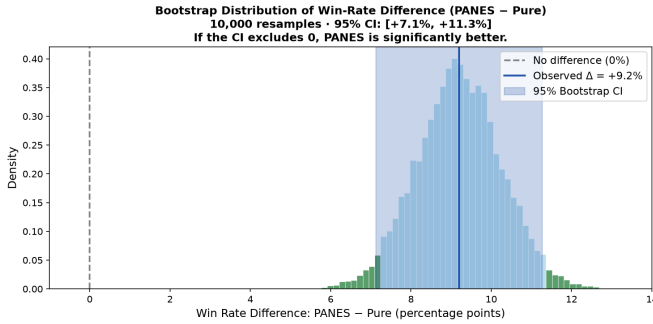


Fig. 5. Distribution of the win-rate difference across 10,000 bootstrap resamples with the 95% Confidence Interval highlighted.

the loss curves behave non-monotonically, which we shall address in the Loss Dynamics subsection.

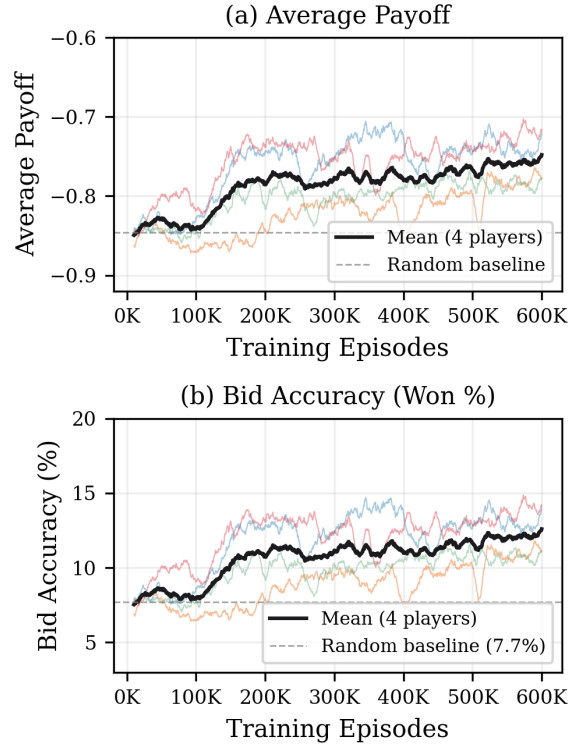


Fig. 6. Convergence of NFSP training across 600K episodes. (a) Average payoff (Averaged using 500 eval games). (b) Bid accuracy(Win%). Each thin colored line represents a player, whereas the thick black line represents the mean across 4 players. The random baseline is shown by the dashed gray line.

TABLE IV
LEARNING PROGRESS AT TRAINING MILESTONES

Episode	Avg Payoff	Won %	RL Loss	SL Loss
50,000	-0.829	8.5	0.86	0.088
100,000	-0.840	8.0	1.31	0.139
200,000	-0.784	10.8	2.27	0.201
400,000	-0.780	11.0	1.71	0.105
600,000	-0.748	12.6	1.61	0.075

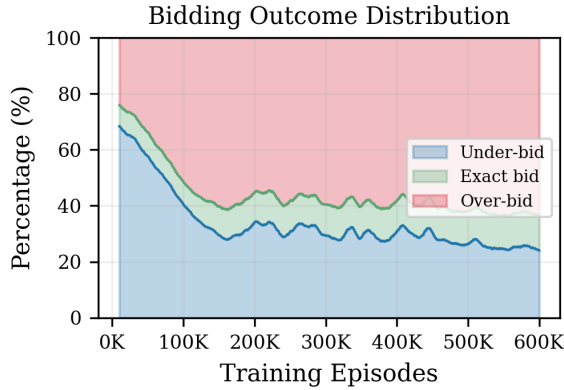


Fig. 7. Evolution of bidding outcome distribution during training. (a) Under-bidding (blue) (b) Over-bidding (red) (c) Exact match (green)

TABLE V
BIDDING BEHAVIOR SHIFT DURING TRAINING (PLAYER 0)

Phase	Won %	Under %	Over %
Early (0–50K)	8.3	64.3	27.5
Late (540K–600K)	12.3	20.4	67.3
Δ	+4.0	−43.9	+39.8

D. Bidding Behavior Evolution

Figure 7 and Table V reveals that under-bidding collapses from $\sim 64\%$ early to $\sim 20\%$ at convergence, while over-bidding rises from $\sim 28\%$ to $\sim 67\%$.

Agents learn aggressive trick-winning before learning to “duck” i.e. deliberately lose tricks once their bid is met. Despite the asymmetric dense reward schedule, over-bidding accounts for $\sim 67\%$ of failures at convergence.

E. Loss Dynamics

The RL loss peaks at ~ 2.4 around episode 250K then settles to ~ 1.5 . The SL loss spikes at 100K–250K as the best response shifts rapidly, then decays to ~ 0.08 , indicating a well-calibrated average policy.

F. Player Symmetry

Table VI shows all four seats converge within 11.2–12.3% bid accuracy (1.1 pp spread). Residual differences reflect positional effects such as the Dealer’s Hook constraint.

G. Prioritized Experience Replay Ablation

We reran training with PER [15] (TD-error priority sampling, $\beta: 0.4 \rightarrow 1.0$). As shown in Table VII, PER reduces final RL loss by 32% (1.01 vs. 1.48) and tightens player spread from 3.2 to 1.2 pp, but yields no significant gain in bid accuracy or payoff—confirming the bottleneck is structural rather than a sampling issue.

From this, we conclude that the primary bottleneck in learning Judgement is not sample efficiency in the replay buffer, but rather a structural credit assignment challenge. Ducking is a core skill in Judgement, and improving the replay

TABLE VI
FINAL PER-PLAYER PERFORMANCE (LAST 60K EPISODES)

Metric	P0	P1	P2	P3
Avg Payoff	−0.754	−0.753	−0.764	−0.776
Won %	12.3	12.3	11.8	11.2
Under %	20.4	28.4	28.6	21.3
Over %	67.3	59.3	59.6	67.5
RL Loss	0.847	1.495	0.946	0.751
SL Loss	0.073	0.090	0.075	0.067

TABLE VII
PER ABLATION: FINAL METRICS (MEAN ACROSS 4 PLAYERS, LAST 60K EPISODES)

Metric	Without PER	With PER
Won %	12.1	11.9
Avg Payoff	−0.757	−0.762
Final RL Loss	1.48	1.01
Final SL Loss	0.082	0.076
Player Spread	3.2 pp	1.2 pp
Peak Won % (P0)	18.8	17.4

sampling distribution does not fundamentally change the fact that one-step bootstrapping limits the agent’s ability to plan for multi-trick avoidance.

VII. LIMITATIONS

A few limitations of the current work deserve mention:

The Ducking Bottleneck: The most persistent learning bottleneck is the agent’s tendency to overbid and win more tricks than declared (accounting for $\sim 67\%$ of failures at convergence). Despite the asymmetric penalties in our dense reward schedule, deliberate trick avoidance remains unresolved, representing the single largest barrier to higher bid accuracy.

Simplified Round and State Representation: NFSP training is confined to single 13-card rounds rather than the full multi-round sequence of varying hand sizes typical in competitive Judgement. Additionally, the MLP network topology cannot natively exploit sequence-dependent features or temporal correlations in players’ cards.

Static Opponent Modeling and Determinization: The PANES agent models opponents using aggregate equilibrium average policies (π_{avg}) rather than dynamically updating beliefs based on in-context play. Furthermore, the MRV determinization does not maintain a probabilistic posterior over hidden opponent cards, reducing MCTS simulation accuracy in high-uncertainty early rounds.

VIII. FUTURE WORK

This work opens several directions for future research:

Hierarchical and Adaptive Rewards: To resolve the ducking bottleneck, future work will explore hierarchical RL architectures that switch between aggressive trick accumulation and defensive avoidance policies once the bid is met, alongside non-linear penalty scaling for over-winning.

Attention-Based Sequence Architectures: Swapping the MLP for Transformer-based architectures will allow dynamic attention over trick histories and seat order, facilitating void inference. Furthermore, extending self-play training across variable hand sizes (1 to 13 cards) will enable learning of round-dependent bidding strategies.

Bayesian Opponent Modeling and Human Play: Integrating particle filters to maintain a probabilistic belief distribution over opponent hands during online search should increase determinization accuracy. Finally, conducting user studies comparing human players against PANES will provide a benchmark for real-world play.

IX. CONCLUSION

This paper introduced PANES, a reinforcement learning framework for the trick-taking game Judgement. By integrating Neural Fictitious Self-Play (NFSP) with Information Set MCTS, PANES addresses the challenges of high non-stationarity, long credit assignment horizons, and vast information asymmetry. Our custom environment models the full complexity of Judgement under realistic rules and Hook constraints.

In self-play, agents successfully transition from simple trick accumulation to exact contract matching, though defensive ducking remains a key challenge. In arena tournaments, wrapping the trained networks in our asymmetric MCTS search shell yields a 22.7% bid accuracy compared to 13.5% for pure NFSP—representing a 68% relative improvement ($p < 0.0001$). This advantage is consistent across all seats, demonstrating that combining offline equilibrium learning with online tree search is an effective approach for imperfect information card games.

REFERENCES

- [1] D. Zha, J. Xie, W. Ma, S. Zhang, X. Lian, X. Hu, and J. Liu, “Douzero: Mastering doudizhu with self-play deep reinforcement learning,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.06135>
- [2] G. Yang, M. Liu, W. Hong, W. Zhang, F. Fang, G. Zeng, and Y. Lin, “Perfectdou: Dominating doudizhu with perfect information distillation,” 2024. [Online]. Available: <https://arxiv.org/abs/2203.16406>
- [3] R. Sutton and A. Barto, “Reinforcement learning: An introduction,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 1054–1054, 1998.
- [4] J. Heinrich and D. Silver, “Deep reinforcement learning from self-play in imperfect-information games,” 03 2016.
- [5] C. Browne, E. Powley, D. Whitehouse, S. Lucas, P. Cowling, P. Rohlfshagen, S. Tavenor, D. Perez Liebana, S. Samothrakis, and S. Colton, “A survey of monte carlo tree search methods,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, pp. 1–43, 03 2012.
- [6] L. Zhang, Y. Chen, W. Wang, Z. Han, S. Li, Z. Pan, and G. Pan, “A monte carlo neural fictitious self-play approach to approximate nash equilibrium in imperfect-information dynamic games,” *Frontiers of Computer Science*, vol. 15, 10 2021.
- [7] D. Zha, K.-H. Lai, Y. Cao, S. Huang, R. Wei, J. Guo, and X. Hu, “Rlcard: A toolkit for reinforcement learning in card games,” 2020. [Online]. Available: <https://arxiv.org/abs/1910.04376>
- [8] G. W. Brown, “Iterative solution of games by fictitious play,” in *Activity Analysis of Production and Allocation*, T. C. Koopmans, Ed. New York: Wiley, 1951.
- [9] J. Robinson, “An iterative method of solving a game,” *Annals of Mathematics*, vol. 54, no. 2, pp. 296–301, 1951. [Online]. Available: <http://www.jstor.org/stable/1969530>
- [10] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:205242740>
- [11] P. Cowling, E. Powley, and D. Whitehouse, “Information set monte carlo tree search,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, pp. 120–143, 06 2012.
- [12] I. Frank and D. Basin, “A theoretical and empirical investigation of search in imperfect information games,” *Theoretical Computer Science*, vol. 252, no. 1, pp. 217–256, 2001, cG’98. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0304397500000839>
- [13] Y. Jing, B. Liu, K. Li, Y. Zang, H. Fu, Q. FU, J. Xing, and J. Cheng, “Opponent modeling with in-context search,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=bGhsbfyg3b>
- [14] A. Y. Ng, D. Harada, and S. J. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proceedings of the Sixteenth International Conference on Machine Learning*, ser. ICML ’99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, p. 278–287.
- [15] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” 2016. [Online]. Available: <https://arxiv.org/abs/1511.05952>
- [16] M. Lanctot, E. Lockhart, J.-B. Lespiau, V. Zambaldi, S. Upadhyay, J. Pérolat, S. Srinivasan, F. Timbers, K. Tuyls, S. Omidshafiei, D. Hennes, D. Morrill, P. Muller, T. Ewalds, R. Faulkner, J. Kramár, B. D. Vylder, B. Saeta, J. Bradbury, D. Ding, S. Borgeaud, M. Lai, J. Schrittwieser, T. Anthony, E. Hughes, I. Danihelka, and J. Ryan-Davis, “OpenSpiel: A framework for reinforcement learning in games,” 2020. [Online]. Available: <https://arxiv.org/abs/1908.09453>
- [17] M. L. Littman, “Markov games as a framework for multi-agent reinforcement learning,” in *International Conference on Machine Learning*, 1994. [Online]. Available: <https://api.semanticscholar.org/CorpusID:8108362>
- [18] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [19] L. Kocsis and C. Szepesvári, “Bandit based monte-carlo planning,” in *Machine Learning: ECML 2006*, J. Fürnkranz, T. Scheffer, and M. Spiliopoulou, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 282–293.